UNIVERSIDAD MAYOR
para espíritus emprendedores
FACULTAD DE CIENCIAS, INGENIERÍA Y TECNOLOGÍA
Escuela de Ingeniería Civil en Computación e Informática
INGT1003-001 · Arquitectura de Desarrollo (Móvil y Web)



Rutia

Hito 2: documento técnico del back end

Integrantes
Carrera, Francisco
Toro, Gerzon

Nombre del equipo: Equipo Rutia

Docente
Carlos Muñoz S.

Santiago, Chile
30 de octubre de 2026

Índice

1. Introducción....1
2. Tecnologías empleadas....1
3. Decisión de arquitectura: monolito en tres capas....2
4. Diseño de la API REST....4
    4.1 Recursos y convenciones....4
    4.2 Tabla de la API....4
    4.3 Ejemplos de petición y respuesta....6
    4.4 Códigos de estado....7
    4.5 Criterio de versionado....8
5. Capas y recorrido de una petición....8
    5.1 Estructura de carpetas por capa....9
    5.2 Recorrido completo: el caso Daniel....10
    5.3 Reglas del generador de rutas....11
6. Persistencia y modelo de datos....12
    6.1 Script de creación reproducible....13
7. Credenciales y configuración....14
    7.1 Contraseñas de los usuarios....15
8. Datos personales y Ley 21.719....15
    8.1 Qué datos trata y para qué....15
    8.2 Derechos del titular en la API....16
    8.3 Medidas técnicas....16
9. Validación, errores y pruebas....17
    9.1 Validación en dos niveles....17
    9.2 Pruebas automáticas con pytest....17
    9.3 Colección de Postman....18
10. Cambios respecto del Hito 1....18
    10.1 Pendientes que el Hito 1 dejó para este hito....20
11. Limitaciones conocidas y trabajo para el Hito 3....20
12. Conclusión....21
13. Bibliografía....22
14. Anexos....24
    Anexo A: evidencia de las pruebas....24
    Anexo B: declaración de uso de inteligencia artificial....25

1. Introducción

En el Hito 1 (Equipo Rutia, 2026), prometimos lo siguiente: "Para el turista de larga distancia al que le cuesta decidir qué visitar en Santiago entre tantas alternativas, Rutia entrega en minutos una ruta con pocos lugares, elegidos según su tiempo, presupuesto e intereses". El problema no es encontrar información, sino decidir entre muchas opciones. Por eso Rutia filtra antes de mostrar y entrega una ruta corta y ordenada.

En este documento describimos el servicio del lado del servidor que construimos en el Hito 2: el motor que sostiene esa promesa. Es un monolito en tres capas, escrito en Python con FastAPI, que guarda sus datos en PostgreSQL. Expone una API REST bajo `/api/v1` para lugares, rutas, usuarios y visitas, y arma la ruta sugerida a partir del tiempo, el presupuesto y los intereses del turista.

Nuestro objetivo es que el lector entienda cómo construimos el servicio y por qué. En la sección 2 justificamos las tecnologías y en la sección 3, la decisión de arquitectura. Las secciones 4 a 9 detallan la API, las capas, la persistencia, las credenciales, los datos personales y las pruebas. En la sección 10 declaramos qué cambió respecto del Hito 1, y las secciones 11 y 12 cierran con las limitaciones y la conclusión.

El código está en la rama principal del repositorio github.com/Sxnr/Rutia. El archivo `README.md` explica cómo levantar el servicio en otro computador.

2. Tecnologías empleadas

Mantenemos las decisiones que firmamos en el Hito 1: Python con FastAPI en el servidor y PostgreSQL como motor de datos (Hito 1, sección 6). El Hito 2 suma las piezas que las conectan y el lugar donde vive la base. La Tabla 1 resume cada elección con su criterio, la alternativa descartada y el costo que aceptamos.

Tabla 1. *Tecnologías del back end: criterio, alternativa y costo asumido.*

Pieza	Elección	Criterio	Alternativa	Costo que asumimos
Lenguaje	Python 3.12 o superior	Lectura clara y una biblioteca estándar que ya incluye el hash de contraseñas (Python Software Foundation, s. f.)	JavaScript con Node.js	El proyecto usa dos lenguajes: Python en el servidor y JavaScript en la app
Framework	FastAPI	Valida cada petición con Pydantic, genera la documentación interactiva desde el código (OpenAPI) e inyecta dependencias como la sesión de base de datos y el idioma (FastAPI, s. f.-a)	Express	Su error de validación por defecto (422) no calza con el criterio del Hito 1: lo reemplazamos por un manejador propio que responde 400 (sección 9)

Pieza	Elección	Criterio	Alternativa	Costo que asumimos
Acceso a datos	SQLAlchemy 2 (ORM)	Cada tabla se declara una vez como clase de Python y las consultas se arman con `select()`, sin pegar texto SQL (SQLAlchemy, s. f.)	SQL escrito a mano	Una biblioteca más que aprender, y el SQL que llega a la base queda menos a la vista
Controlador	psycopg 3	Adaptador de PostgreSQL para Python; SQLAlchemy lo usa por debajo (Psycopg, s. f.)	—	Ninguno relevante
Motor de datos	PostgreSQL	Claves foráneas, restricciones `CHECK` y borrado en cascada mantienen consistentes usuarios, rutas y visitas (PostgreSQL Global Development Group, s. f.)	MongoDB	Diseñar tablas y relaciones antes de programar
Alojamiento	Supabase, plan gratuito	PostgreSQL administrado, que compartimos los dos y accesible desde cualquier computador para la demostración, sin costo	PostgreSQL en un notebook	500 MB de límite y pausa del proyecto tras una semana sin uso; el plan Pro cuesta desde USD 25 al mes (Supabase, s. f.-b)
Contraseñas	PBKDF2-HMAC-SHA256 con 600.000 iteraciones	Viene en `hashlib`, sin dependencias externas; 600.000 iteraciones es el mínimo que OWASP indica para PBKDF2 (OWASP Foundation, s. f.)	Argon2id	OWASP prefiere Argon2id: PBKDF2 resiste menos ataques con hardware especializado
Pruebas	pytest y Postman	pytest prueba el generador sin base y la API contra una base real; Postman guarda las peticiones de la demostración	Pruebas manuales	Mantener las pruebas al día con cada cambio

La conexión a Supabase usa el *Session pooler* en el puerto 5432, porque funciona en redes IPv4; la conexión directa usa IPv6 (Supabase, s. f.-a). Las pruebas automáticas y el plan B de la demostración usan PostgreSQL local, creado con el mismo script.

3. Decisión de arquitectura: monolito en tres capas

Decisión. Mantenemos un monolito modular organizado en tres capas (presentación, negocio y datos), que corre como un único servicio FastAPI conectado a una única base PostgreSQL. No elegimos esta arquitectura porque sea la única válida, sino porque corresponde al tamaño y a la carga que podemos justificar hoy: Rutia tiene dos integrantes activos, un catálogo inicial de nueve lugares, una API que respalda la demostración del Hito 2 y todavía no cuenta con métricas de tráfico de producción ni pruebas de carga que indiquen la necesidad de distribuir el sistema.

Carga y capacidad actual. El arranque documentado ejecuta `uvicorn app.main:app --reload`, sin workers adicionales; por tanto, en desarrollo la aplicación corre en un solo proceso. Las rutas HTTP, los servicios de negocio y los repositorios se ejecutan dentro de ese mismo servicio. La capa de datos mantiene un pool de SQLAlchemy de cinco conexiones y permite hasta cinco adicionales cuando aumenta la demanda (`backend/app/core/database.py`). Este límite fija cuántas conexiones abrimos hacia PostgreSQL, pero no es por sí solo una medición de usuarios concurrentes ni una garantía de rendimiento. Todavía no contamos con métricas de tráfico de producción ni con resultados de pruebas de carga; con la evidencia disponible, preferimos un despliegue simple antes que introducir servicios separados (FastAPI, s. f.-b).

Equipo y operación. Con dos personas, un único despliegue reduce el trabajo de coordinar versiones, variables de entorno, registros, monitoreo y fallos entre servicios. También permite conservar una sola configuración (`.env`), un solo punto de entrada y una base compartida. La separación en carpetas no es decorativa: presentación recibe HTTP y valida el formato; negocio aplica reglas y genera rutas sin depender de FastAPI; datos concentra las consultas y la persistencia. Así mantenemos límites internos claros sin asumir el costo operativo de una arquitectura distribuida.

Costo de alojamiento. Supabase publica un plan Free de USD 0 con 500 MB de base de datos por proyecto, que se pausa tras una semana de poca actividad; el plan Pro comienza en USD 25 al mes (Supabase, s. f.-b, s. f.-c; páginas consultadas el 29 de octubre y el 1 de noviembre de 2026). Para el catálogo y las pruebas actuales, el plan gratuito evita un costo fijo: aceptamos la pausa y mantenemos PostgreSQL local como respaldo para pruebas y demostraciones. Si el uso exige disponibilidad continua, más almacenamiento o más recursos, el costo del plan Pro tendrá que entrar al presupuesto.

Complejidad de despliegue. Con el monolito, configuramos una URL de base de datos, levantamos un servicio y revisamos un registro de ejecución. Separar, por ejemplo, usuarios, catálogo y generador de rutas en procesos independientes exigiría definir la comunicación entre servicios, despliegues coordinados y más configuración y observabilidad. Además, los servicios separados no comparten la memoria del proceso: hoy el límite de intentos de inicio de sesión se guarda en memoria y se reinicia junto con el servidor (`backend/app/business/limite_intentos.py`). Si ejecutáramos varias instancias, cada una llevaría un contador distinto; habría que moverlo a PostgreSQL o a un servicio compartido como Redis antes de escalar horizontalmente. La documentación de Microsoft también advierte que los microservicios agregan complejidad en la comunicación entre servicios, la consistencia de los datos y las pruebas de sus dependencias (Microsoft, s. f.).

Cuándo revisaríamos la decisión. Reevaluaremos el monolito si las mediciones muestran que un único proceso o el pool de conexiones limita la respuesta; si el generador necesita escalar de forma independiente por su carga de cálculo; si aparecen equipos distintos que necesiten desplegar módulos sin coordinarse; o si la disponibilidad exige redundancia y despliegue independiente. Antes de dividir en microservicios, podemos aumentar los

workers o los recursos del mismo servicio y trasladar el limitador a un almacenamiento compartido. Revisaremos la decisión con métricas de latencia, errores, uso de CPU y memoria, conexiones y volumen de peticiones, no solo por un crecimiento hipotético.

4. Diseño de la API REST

La API expone los recursos del dominio de Rutia bajo `/api/v1`. Lugares, rutas guardadas y usuarios tienen las cuatro operaciones: crear, leer, actualizar y borrar. La Tabla 2 (sección 4.2) lista todas las rutas; la misma tabla está en el repositorio, en `docs/hito-2/API.md`, y la documentación interactiva que genera FastAPI queda en `http://localhost:8000/docs`.

4.1 Recursos y convenciones

- **Recursos:** `lugares` (con sus `categorias`, de solo lectura), `rutas`, `usuarios` y `visitas`.
- **Rutas:** sustantivos en plural para las colecciones (`/lugares`) y el id en la ruta para un elemento (`/lugares/{id}`). Lo que pertenece a un usuario cuelga de él: `/usuarios/{id}/rutas` y `/usuarios/{id}/visitas`.
- **Verbos:** GET lee, POST crea, PUT reemplaza y DELETE borra. Al crear, la respuesta es 201 con la cabecera `Location` del recurso nuevo.
- **Acciones que calculan sin guardar:** `/rutas/generar` y `/rutas/recalcular` usan POST y responden 200. Reciben las preferencias o una lista de lugares en el cuerpo, que no cabe bien en una URL, y no crean nada.
- **Formato:** JSON en cuerpos y respuestas. El idioma se elige con la cabecera `Accept-Language` (`es` por defecto o `en`) y cambia nombres, descripciones, horarios y mensajes.
- **Paginación:** `GET /lugares` recibe `limit` (1 a 100) y `offset`, y responde `{datos, total, limit, offset}`.
- **Errores:** un solo formato para todos, `{ "error": { "codigo", "mensaje", "detalles" } }` (sección 4.4).

4.2 Tabla de la API

Tabla 2. *Rutas de la API v1 con parámetros, código de éxito y códigos de error.*

Recurso	Verbo y ruta	Parámetros o cuerpo	Éxito	Errores
Salud	`GET /health`	—	200	503 base no disponible
Categorías	`GET /categorias`	—	200	—
Lugares	`GET /lugares`	Query: `categoria`, `costo_max`, `duracion_max`, `limit`, `offset`	200 página	400 · 422 categoría inexistente
Lugares	`GET /lugares/{id}`	—	200	400 · 404

Recurso	Verbo y ruta	Parámetros o cuerpo	Éxito	Errores
Lugares	`POST /lugares`	Cuerpo: lugar completo (4.3)	201 + `Location`	400 · 409 slug repetido · 422 categoría inexistente
Lugares	`PUT /lugares/{id}`	Cuerpo: lugar completo, lo reemplaza	200	400 · 404 · 409 · 422
Lugares	`DELETE /lugares/{id}`	—	204	400 · 404 · 409 está en una ruta guardada
Rutas	`POST /rutas/generar`	`tiempo_horas` (hasta 8), `presupuesto_clp`, `intereses[]`, `cantidad?`, `usuario_id?`, `incluir_visitados?`	200 ruta sugerida, no se guarda	400 · 404 usuario · 422 categoría inexistente
Rutas	`POST /rutas/recalcular`	`lugares[]` en el nuevo orden, `tiempo_horas?`, `presupuesto_clp?`	200 traslados y totales nuevos, no se guarda	400 · 422 lugar inexistente o repetido
Rutas	`POST /rutas`	`usuario_id`, `nombre`, `lugares[]`, `tiempo_horas?`, `presupuesto_clp?`	201 + `Location`	400 · 404 usuario · 422 lugar inexistente o repetido
Rutas	`GET /usuarios/{id}/rutas`	—	200 lista resumida	400 · 404
Rutas	`GET /rutas/{id}`	—	200 con paradas y lugares	400 · 404
Rutas	`PUT /rutas/{id}`	`nombre`, `lugares[]`, `tiempo_horas?`, `presupuesto_clp?`	200 con totales recalculados	400 · 404 · 422
Rutas	`DELETE /rutas/{id}`	—	204	400 · 404
Usuarios	`POST /usuarios`	`nombre`, `email`, `password`, `idioma`	201 + `Location`	400 · 409 correo registrado
Usuarios	`POST /usuarios/login`	`email`, `password`	200 datos públicos	400 · 401 · 429
Usuarios	`GET /usuarios/{id}`	—	200	400 · 404
Usuarios	`PUT /usuarios/{id}`	`nombre`, `email`, `idioma`, `password`	200	400 · 401 · 404 · 409 correo de otro usuario · 429

Recurso	Verbo y ruta	Parámetros o cuerpo	Éxito	Errores
Usuarios	`DELETE /usuarios/{id}`	`password`	204, borra también sus rutas y visitas	400 · 401 · 404 · 429
Usuarios	`POST /usuarios/{id}/datos`	`password`	200 archivo JSON descargable	400 · 401 · 404 · 429
Visitas	`POST /usuarios/{id}/visitas`	`lugar_id`, `ruta_id?`	201	400 · 404 usuario · 422 lugar o ruta inválidos
Visitas	`GET /usuarios/{id}/visitas`	—	200, más recientes primero	400 · 404
Visitas	`DELETE /visitas/{id}`	—	204	400 · 404

El signo `?` marca un campo opcional. Todas las rutas llevan el prefijo `/api/v1`.

4.3 Ejemplos de petición y respuesta

Crear un lugar · `POST /api/v1/lugares` → 201. Es la primera petición de la demostración:

```
{
  "slug": "barrio-lastarria",
  "categoria": "patrimonio",
  "nombre_es": "Barrio Lastarria",
  "nombre_en": "Lastarria Neighborhood",
  "descripcion_es": "Barrio patrimonial con cafés, librerías y galerías.",
  "descripcion_en": "Heritage neighborhood with cafés, bookshops and galleries.",
  "comuna": "Santiago",
  "latitud": -33.4372,
  "longitud": -70.6386,
  "costo_clp": 0,
  "duracion_min": 45,
  "horario_es": "Lun-Dom 10:00-20:00",
  "horario_en": "Mon-Sun 10:00-20:00",
  "contenido_rotativo": false
}
```

Generar una ruta · `POST /api/v1/rutas/generar` → 200. Es el caso de uso de Daniel del Hito 1 (sección 5.3): 4 horas, $15.000, museos y restaurantes.

```
{
  "tiempo_horas": 4,
  "presupuesto_clp": 15000,
  "intereses": ["museos", "restaurantes"]
}
```

Respuesta, resumida a los campos principales de cada lugar:


```
{
  "tiempo_disponible_min": 240,
  "presupuesto_clp": 15000,
  "cantidad_objetivo": 4,
  "duracion_total_min": 229,
  "costo_total_clp": 4590,
  "paradas": [
    { "orden": 1, "traslado_min": 0,
      "lugar": { "id": 1, "nombre": "Museo Nacional de Bellas Artes",
                 "costo_clp": 0, "duracion_min": 60 } },
    { "orden": 2, "traslado_min": 3,
      "lugar": { "id": 2, "nombre": "Museo de Arte Contemporáneo",
                 "costo_clp": 0, "duracion_min": 50 } },
    { "orden": 3, "traslado_min": 9,
      "lugar": { "id": 3, "nombre": "Emporio La Rosa",
                 "costo_clp": 4590, "duracion_min": 40 } },
    { "orden": 4, "traslado_min": 22,
      "lugar": { "id": 9, "nombre": "Centro Cultural Gabriela Mistral (GAM)",
                 "costo_clp": 0, "duracion_min": 45 } }
  ],
  "mensaje": null
}
```


La ruta usa 229 de los 240 minutos y $4.590 de los $15.000. Si caben menos lugares de los buscados, `mensaje` lo explica; si no cabe ninguno, `paradas` llega vacía con una sugerencia.

Formato inválido → 400, con un detalle por campo:


```
{
  "error": {
    "codigo": "VALIDACION",
    "mensaje": "Revisa el formato de los datos enviados.",
    "detalles": [
      { "campo": "tiempo_horas", "mensaje": "Debe ser mayor que 0." },
      { "campo": "presupuesto_clp", "mensaje": "Debe ser mayor o igual a 0." },
      { "campo": "intereses", "mensaje": "Debe tener al menos 1 elemento(s)." }
    ]
  }
}
```


Regla de negocio no cumplida → 422. El formato es correcto, pero la categoría no existe:


```
{
  "error": {
    "codigo": "CATEGORIA_NO_EXISTE",
    "mensaje": "La categoría 'playas' no existe. Usa una de: museos, parques, patrimonio,
      restaurantes.",
    "detalles": []
  }
}
```


4.4 Códigos de estado

Cada error responde con el código que corresponde y nunca con 200. La Tabla 3 muestra cuándo se usa cada uno y qué capa lo decide.

Tabla 3. *Códigos de estado de la API y capa que los decide.*

Código	Cuándo	Capa que lo decide
200 · 201 · 204	Éxito: leído o actualizado · creado, con `Location` · borrado, sin cuerpo	—
400	Formato inválido: falta un campo, tipo incorrecto, número fuera de rango, JSON mal escrito o campo no permitido	Presentación (Pydantic)
401	Correo o contraseña incorrectos	Negocio
404	El recurso no existe	Negocio
409	Choca con lo guardado: slug o correo repetido, lugar usado en una ruta	Negocio
422	Regla de negocio no cumplida: categoría inexistente, lugar repetido o inexistente en una ruta, ruta de otro usuario	Negocio
429	5 contraseñas incorrectas en 15 minutos; la cabecera `Retry-After` indica los segundos de espera	Negocio
500	Error inesperado: mensaje genérico, el detalle queda solo en el registro del servidor	Presentación
503	`GET /health` no logra conectarse a la base	Presentación, con la respuesta de la capa de negocio

La diferencia entre 400 y 422 sigue la definición de HTTP: 422 indica que el servidor entendió el formato y la sintaxis de la petición, pero no pudo procesar lo que pedía (Mozilla Developer Network [MDN], s. f.).

4.5 Criterio de versionado

La versión va en la ruta: `/api/v1`. Un cambio **compatible** se publica en la misma versión: agregar una ruta (como `/rutas/recalcular`), un campo opcional en la petición (como `cantidad`) o un campo nuevo en la respuesta (como `cantidad_objetivo`). Un cambio **incompatible** se publica como `/api/v2` y convive con `v1` hasta que los clientes migren: quitar o renombrar un campo, cambiar su tipo o su significado, o cambiar un código de estado del que dependa un cliente.

El paso de 422 a 400 para los errores de formato, la descarga de datos de GET a POST y el tope de 8 horas los hicimos dentro de `v1` porque ningún cliente estaba publicado: la app móvil sigue en desarrollo. Desde la primera versión publicada de la app, aplicaremos la regla sin excepciones.

5. Capas y recorrido de una petición

El servidor es un monolito en tres capas, como define el Hito 1 (sección 5.2): presentación, negocio y datos. Cada capa vive en su propia carpeta y solo llama a la de abajo (Figura 1).

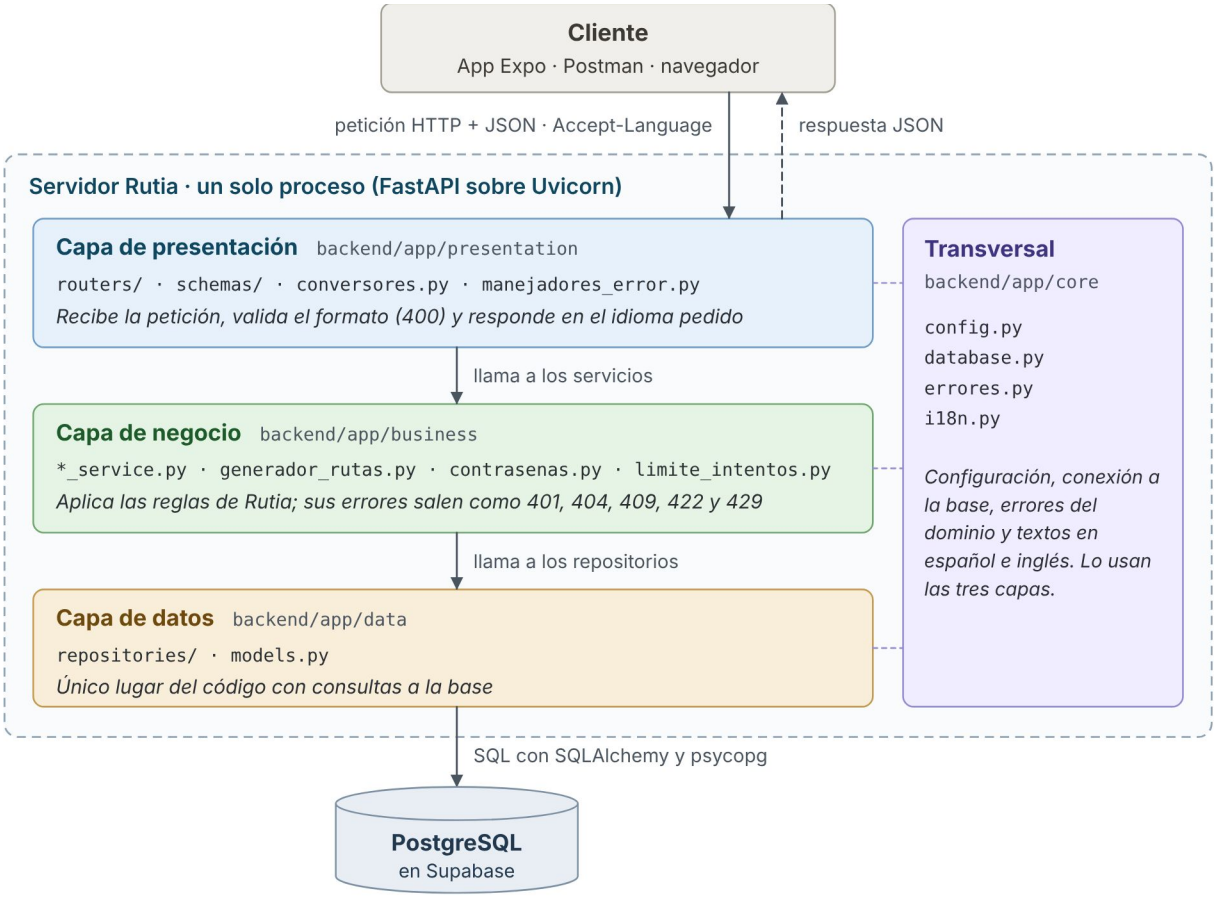


Figura 1. *Capas del servidor de Rutia y su relación con el cliente y la base de datos. Elaboración propia.*

5.1 Estructura de carpetas por capa

```
backend/
├── app/
│   ├── main.py                  Arranque: rutas, CORS, /fotos y manejo de errores
│   ├── core/                    TRANSVERSAL
│   │   ├── config.py            Lee el .env de la raíz del repositorio
│   │   ├── database.py          Motor de SQLAlchemy y una sesión por petición
│   │   ├── errores.py           Errores del dominio: NoEncontrado, Conflicto…
│   │   └── i18n.py              Mensajes en español e inglés
│   ├── presentation/            CAPA DE PRESENTACIÓN
│   │   ├── routers/             lugares · rutas · usuarios · visitas · salud
│   │   ├── schemas/             Contratos de entrada y salida (Pydantic)
│   │   ├── dependencias.py      Entrega a cada ruta su servicio y el idioma
│   │   ├── conversores.py       Modelo → JSON en el idioma pedido
│   │   └── manejadores_error.py Error → código HTTP y formato común
│   ├── business/                CAPA DE NEGOCIO
│   │   ├── *_service.py         lugares · rutas · usuarios · visitas · salud
│   │   ├── generador_rutas.py   Algoritmo de la ruta (funciones puras)
│   │   ├── contrasenas.py       Hash PBKDF2 de contraseñas
│   │   └── limite_intentos.py   Bloqueo tras 5 contraseñas incorrectas
│   └── data/                    CAPA DE DATOS
│       ├── models.py            Tablas como clases de SQLAlchemy
│       └── repositories/        Un repositorio por tabla: las únicas consultas
├── scripts/                     crear_base.py · cargar_catalogo.py
└── tests/                       test_generador.py · test_api.py · test_api_cuenta.py
```

Tabla 4. *Qué hace y qué no hace cada capa.*

Capa	Hace	No hace
Presentación	Recibe HTTP, valida el formato con los `schemas`, llama a un servicio y convierte la respuesta al idioma pedido	No escribe SQL ni decide qué lugares van en una ruta
Negocio	Aplica las reglas: candidatos, revisitas, cantidad, orden, totales, contraseñas y límites	No importa FastAPI: lanza errores del dominio y presentación los traduce a códigos HTTP
Datos	Consulta y guarda con `select()` y `commit()`	No decide nada: entrega los datos listos para usar

La Tabla 4 resume lo que hace y lo que no hace cada capa, y esas reglas se pueden comprobar en el código. Estas búsquedas no encuentran nada:

```
grep -rn "fastapi" backend/app/business backend/app/data --include=*.py
grep -rnE "select\(|\.execute\(|\.commit\(" \
     backend/app/presentation backend/app/business --include=*.py
```

Hay dos cruces declarados. `dependencias.py` abre una sesión de base por petición y se la entrega al servicio, sin consultar con ella. `conversores.py` importa las clases del modelo solo para anotar tipos: lee sus atributos para armar el JSON, pero no consulta ni guarda.

5.2 Recorrido completo: el caso Daniel

Estos pasos siguen la petición de Daniel del Hito 1 desde que llega al servidor hasta la base de datos y de vuelta. La Figura 2 los resume en un diagrama de secuencia.

1. **Llega la petición.** El cliente envía `POST /api/v1/rutas/generar` con 4 horas, $15.000, museos y restaurantes, y la cabecera `Accept-Language`.
2. **Presentación valida el formato.** FastAPI dirige la petición a `generar_ruta` en `routers/rutas.py` y valida el cuerpo contra `GenerarRutaEntrada` (`schemas/rutas.py`): más de 0 y hasta 8 horas, de $0 a $1.000.000, de 1 a 4 intereses y una cantidad opcional de 2 a 5. Si algo falla, `manejadores_error.py` responde 400 y la petición no sigue. `dependencias.py` entrega a la función el idioma y un `RutasServicio`.
3. **Negocio revisa las reglas.** `RutasServicio.generar` (`business/rutas_service.py`) convierte 4 horas en 240 minutos y pide a `CategoriasRepositorio` confirmar las categorías. Si una no existe, lanza `DatosInvalidos` y el cliente recibe 422.
4. **Datos consulta la base.** `LugaresRepositorio.buscar_candidatos` (`data/repositories/lugares_repo.py`) arma con `select()` la consulta de los lugares activos de esas categorías que, por sí solos, cuestan $15.000 o menos y duran 240 minutos o menos. PostgreSQL devuelve 5 lugares, que llegan a negocio como objetos `Lugar` (`data/models.py`).
5. **Negocio arma la ruta.** Para 240 minutos se sugieren 4 lugares. `generador_rutas.generar_ruta` parte por el más barato y agrega siempre el más cercano que cabe en el tiempo y el dinero que quedan (sección 5.3). Resultado: 4 paradas, 229 minutos y $4.590.

6. **Presentación responde.** `conversores.ruta_generada` elige nombres y descripciones en el idioma pedido y arma la URL de cada foto. FastAPI responde 200 con el JSON de la sección 4.3.

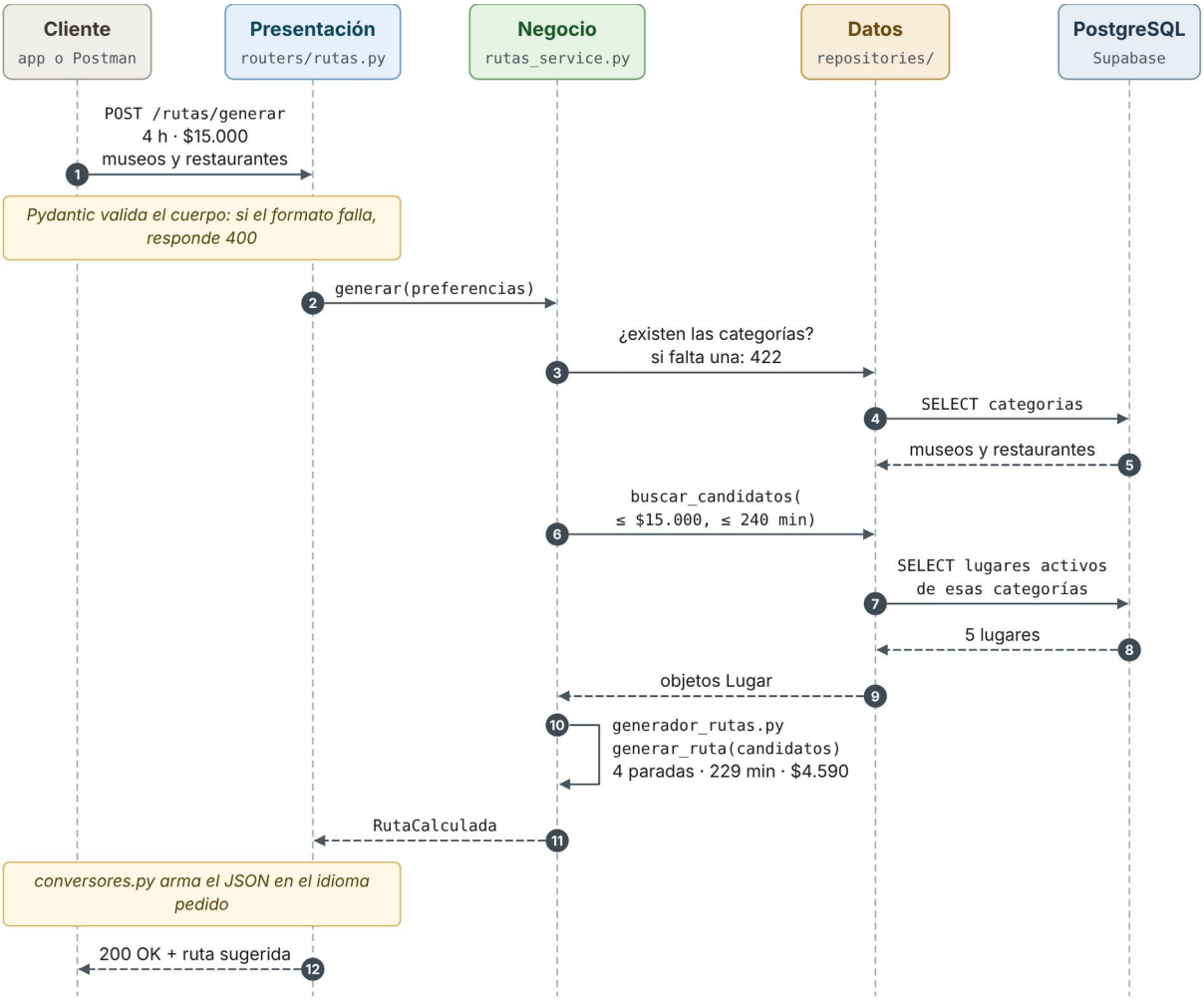


Figura 2. *Recorrido de `POST /api/v1/rutas/generar` por las tres capas. Elaboración propia.*

5.3 Reglas del generador de rutas

El generador es la regla central de la promesa y vive completo en la capa de negocio. Son funciones puras: reciben lugares y números y devuelven una ruta, así que se prueban sin servidor ni base.

1. **Candidatos.** Solo lugares activos de los intereses elegidos que, por sí solos, caben en el tiempo y en el presupuesto.
2. **Revisitas.** Si viene `usuario_id`, se descartan los lugares ya visitados, salvo los de contenido rotativo, como exposiciones o carta de temporada. Con `incluir_visitados` no se descarta ninguno.
3. **Cantidad.** La que pide el turista, de 2 a 5. Si no la indica, se sugiere según su tiempo: hasta 2 horas, 2 lugares; hasta 3 horas, 3; hasta 5 horas, 4; más de 5 horas, 5.

4. **Orden.** Primero el lugar más barato (en empate, el de menor id) y después, siempre, el más cercano que todavía cabe en el tiempo y en el dinero que quedan.
5. **Traslado estimado a pie.** Distancia en línea recta entre dos lugares (fórmula de Haversine) multiplicada por 1,3, porque las calles no son rectas, y recorrida a 4 km/h: unos 19,5 minutos por kilómetro en línea recta.

Si caben menos lugares de los buscados, la respuesta lo explica en `mensaje`. Los totales siempre los calcula el servidor con los datos de la base, nunca el cliente, como exige el Hito 1 (sección 5.2).

6. Persistencia y modelo de datos

Los datos viven en PostgreSQL, en seis tablas relacionadas (Figura 3 y Tabla 5). Un solo script, `db/schema.sql`, crea la base desde cero, y otro carga el catálogo de lugares. Cualquier integrante puede reconstruirla con un comando.

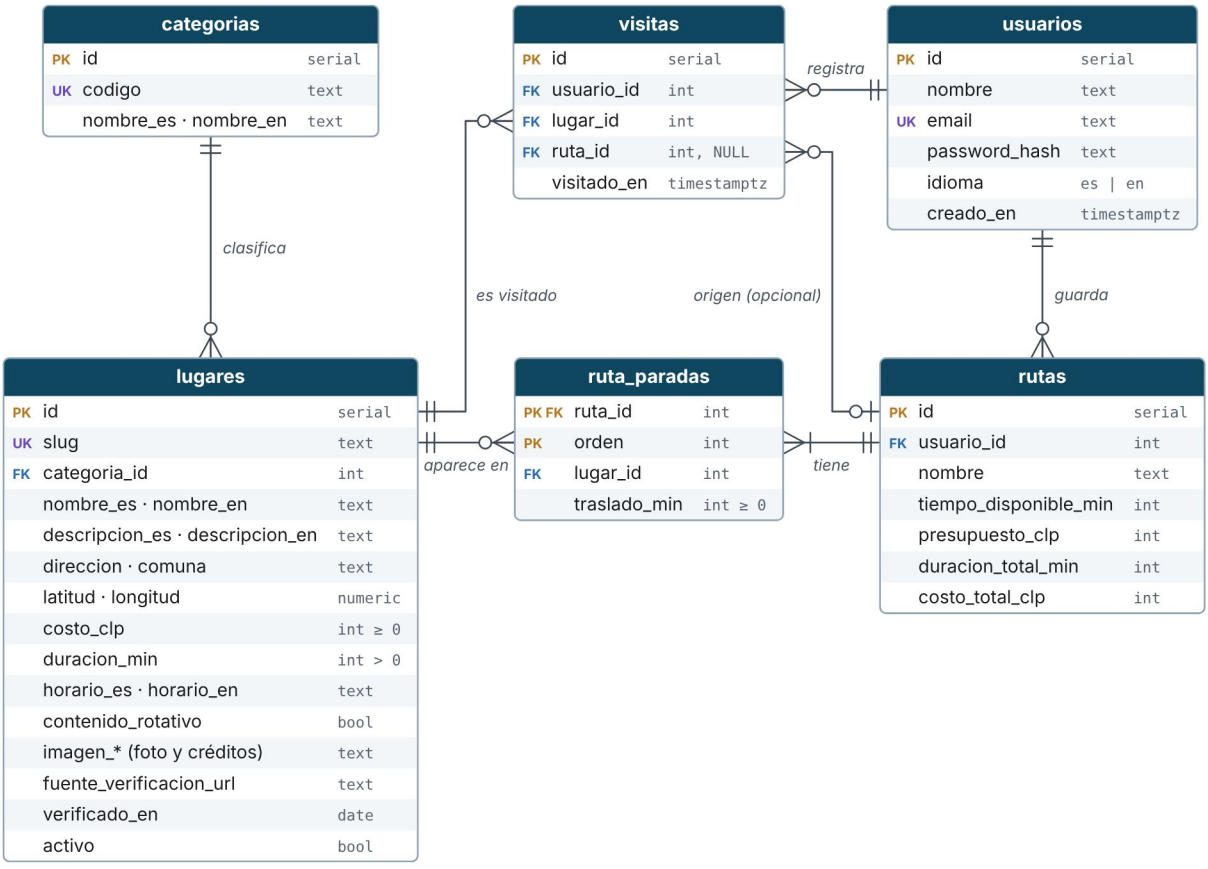


Figura 3. *Modelo entidad-relación de Rutia con tipos, claves y relaciones. Muestra las columnas principales; el detalle completo está en `db/schema.sql`. Elaboración propia.*

Tabla 5. *Tablas, claves y reglas de integridad.*

Tabla	Qué guarda	Clave primaria	Claves foráneas (al borrar)	Reglas
`categorias`	Los 4 intereses: museos, parques, patrimonio y restaurantes	`id` SERIAL	—	`codigo` único; se insertan en el mismo script
`lugares`	Catálogo bilingüe de puntos de interés	`id` SERIAL	`categoria_id` → `categorias`	`slug` único; latitud y longitud NUMERIC(9,6) con rango válido; `costo_clp` ≥ 0; `duracion_min` > 0
`usuarios`	Turistas registrados	`id` SERIAL	—	`email` único y en minúsculas; `idioma` solo `es` o `en`; la contraseña como hash
`rutas`	Rutas que el turista guardó	`id` SERIAL	`usuario_id` → `usuarios` (CASCADE)	Totales calculados por el servidor; tiempo > 0 y presupuesto ≥ 0
`ruta_paradas`	Lugares de cada ruta, en orden	(`ruta_id`, `orden`)	`ruta_id` → `rutas` (CASCADE); `lugar_id` → `lugares` (RESTRICT)	Un lugar aparece una sola vez por ruta; `traslado_min` ≥ 0
`visitas`	Lugares marcados como visitados	`id` SERIAL	`usuario_id` → `usuarios` (CASCADE); `lugar_id` → `lugares` (CASCADE); `ruta_id` → `rutas` (SET NULL)	Se permiten varias visitas al mismo lugar: el turista puede volver

Las reglas de borrado protegen la consistencia sin código extra (PostgreSQL Global Development Group, s. f.):

- **CASCADE** en los datos del usuario: al eliminar una cuenta, la base borra sus rutas, paradas y visitas (sección 8).
- **RESTRICT** en los lugares usados por una ruta guardada: la base impide borrarlos y la API responde 409.
- **SET NULL** en la ruta de una visita: si se borra la ruta, la visita se conserva.

Cuatro índices aceleran las consultas frecuentes: lugares por categoría, lugares por costo y duración (la búsqueda de candidatos), rutas por usuario y visitas por usuario y lugar (la regla de revisitas).

6.1 Script de creación reproducible

Desde la carpeta `backend/`, con el `.env` configurado:

```
python -m scripts.crear_base        # pide confirmación: borra y vuelve a crear todo
```

El script ejecuta `db/schema.sql`, que borra las tablas, las crea y agrega las 4 categorías. Después carga los 9 lugares de `db/catalogo/lugares.csv`. Antes de cargar,

`scripts/cargar_catalogo.py` valida cada fila con las mismas reglas de la API: columnas, categoría, coordenadas y montos. La carga se puede repetir sin duplicar, porque un lugar con el mismo `slug` se actualiza. Cada lugar guarda la página donde se verificaron su dirección, horario y costo, y la fecha de esa verificación. El mismo esquema también se puede aplicar con `psql "$DATABASE_URL" -f db/schema.sql`.

Las fotos de los lugares son archivos en `db/catalogo/fotos`, de Wikimedia Commons con licencia libre. Cada lugar guarda autor, licencia y fuente de su foto, y el backend las sirve en `/fotos`.

Las clases de `data/models.py` reflejan este esquema para SQLAlchemy. El script SQL es la fuente de verdad: un cambio de esquema se aplica recreando la base, porque el proyecto todavía no usa migraciones.

7. Credenciales y configuración

Ninguna credencial está en el código ni en el repositorio. Viven en un archivo `.env` en la raíz, que git ignora; el repositorio solo trae `.env.example`, con todas las variables y valores de ejemplo (Tabla 6).

Tabla 6. *Variables de configuración del backend.*

Variable	Obligatoria	Para qué sirve	Valor de ejemplo
`DATABASE_URL`	Sí	Conexión a PostgreSQL	`postgresql://postgres.TU_PROYECTO:TU_CONTRASEÑA@aws-0-sa-east-1.pooler.supabase.com:5432/postgres?sslmode=require`
`PUBLIC_BASE_URL`	No	Dirección con que los clientes ven el backend; arma la URL de las fotos	`http://localhost:8000`
`CORS_ORIGINS`	No	Orígenes web que pueden llamar a la API	`http://localhost:8081,http://localhost:5173`
`LOG_LEVEL`	No	Detalle del registro del servidor	`INFO`
`TEST_DATABASE_URL`	Solo para pytest	Base que las pruebas borran y recrean	`postgresql://usuario:clave@localhost:5432/rutia_pruebas`

Así se configura y se protege:

1. Quien levanta el servicio copia `.env.example` como `.env` en la raíz y completa `DATABASE_URL` con la cadena del *Session pooler* de Supabase.
2. Al arrancar, `core/config.py` lee el `.env` con python-dotenv. Las variables definidas en el sistema tienen prioridad, lo que sirve para un despliegue.
3. Si falta `DATABASE_URL`, el servidor no arranca y explica qué archivo copiar.
4. `.gitignore` excluye `.env`, así que `git status` nunca lo muestra. La contraseña de la base solo existe en el computador de quien levanta el servicio.

5. Si una credencial llegara a subirse, se cambia en Supabase: borrarla del repositorio no basta, porque queda en el historial.
6. Las pruebas exigen que `TEST_DATABASE_URL` sea distinta de `DATABASE_URL`: si son iguales, `tests/conftest.py` se niega a correr para no borrar la base de la demostración.

La conexión con Supabase va cifrada (`sslmode=require`).

7.1 Contraseñas de los usuarios

La API nunca guarda ni devuelve una contraseña. `business/contrasenas.py` guarda solo un hash con el formato `pbkdf2_sha256$600000$sal$hash`:

- **Sal aleatoria por usuario** de 16 bytes, con `secrets.token_bytes`: dos contraseñas iguales producen hashes distintos.
- **600.000 iteraciones de PBKDF2-HMAC-SHA256**, el mínimo que OWASP indica para este algoritmo (OWASP Foundation, s. f.), con la función `hashlib.pbkdf2_hmac` de la biblioteca estándar (Python Software Foundation, s. f.).
- **Comparación en tiempo constante** con `hmac.compare_digest`.
- **El mismo error y el mismo tiempo de respuesta** si el correo no existe o la contraseña es incorrecta, para no revelar qué correos están registrados.
- **Límite de intentos:** tras 5 contraseñas incorrectas para un correo en 15 minutos, contando login, corrección, descarga y eliminación de la cuenta, la API responde 429.

Las respuestas de usuarios nunca incluyen el hash, y una prueba automática lo verifica.

8. Datos personales y Ley 21.719

La Ley 21.719 regula la protección y el tratamiento de los datos personales en Chile y entra en plena vigencia el 1 de diciembre de 2026 (Ley N.° 21.719, 2024). Rutia trata pocos datos personales (Tabla 7) y ya permite ejercer desde la API los derechos de acceso, rectificación, supresión y portabilidad (Tabla 8).

8.1 Qué datos trata y para qué

Tabla 7. *Datos personales que guarda Rutia, su finalidad y por qué son necesarios.*

Dato	Finalidad	Por qué es necesario
Nombre	Mostrar el perfil	Basta un nombre o un apodo: no se pide apellido ni RUT
Correo electrónico	Iniciar sesión e identificar la cuenta	Es el identificador de la cuenta; Rutia no envía correos
Hash de la contraseña	Comprobar la identidad	Nunca se guarda la contraseña en texto plano
Idioma	Responder en español o inglés	Solo `es` o `en`

Dato	Finalidad	Por qué es necesario
Rutas guardadas	Que el turista siga su ruta durante el viaje	Contienen lugares del catálogo, no la ubicación del turista
Visitas (lugar y fecha)	No volver a sugerir lugares ya visitados	Solo se registran cuando el turista las marca

Por minimización, Rutia **no** pide ubicación GPS, teléfono, fecha de nacimiento, nacionalidad ni datos de pago. La ruta se arma con el tiempo, el presupuesto y los intereses, que no identifican a nadie.

8.2 Derechos del titular en la API

Tabla 8. *Derechos implementados y la ruta que los atiende.*

Derecho	Ruta	Cómo funciona
Acceso	`GET /usuarios/{id}`, `/usuarios/{id}/rutas` y `/usuarios/{id}/visitas`	Muestra el perfil, las rutas guardadas y las visitas
Rectificación	`PUT /usuarios/{id}`	Pide la contraseña y corrige nombre, correo e idioma; si el correo es de otra cuenta, responde 409
Supresión	`DELETE /usuarios/{id}`	Pide la contraseña y borra la cuenta; la base borra en cascada sus rutas, paradas y visitas
Portabilidad	`POST /usuarios/{id}/datos`	Pide la contraseña y entrega perfil, rutas y visitas en un archivo JSON descargable, sin el hash de la contraseña

Si el usuario pide eliminar sus datos, envía su contraseña a `DELETE /usuarios/{id}`. La API responde 204, y desde ese momento la cuenta no existe: su perfil y sus rutas responden 404 y el login falla. Pedir la contraseña para corregir, descargar o eliminar evita que alguien actúe sobre una cuenta ajena conociendo solo su id. Las pruebas automáticas verifican todo este flujo.

8.3 Medidas técnicas

- **Cifrado de contraseñas:** hash PBKDF2 con sal aleatoria (sección 7.1).
- **Control de acceso:** corregir, descargar y eliminar los datos de la cuenta exigen la contraseña, y tras 5 fallos se bloquean por 15 minutos. **Limitación declarada:** sin tokens de sesión, las demás rutas confían en el `usuario_id` que envía el cliente, así que quien conozca un id puede ver el perfil, las rutas y las visitas de esa cuenta. En el Hito 3 cada petición llevará un token firmado y el servidor solo entregará los datos de su dueño.
- **Registro de accesos:** el servidor muestra en consola cada petición con la dirección IP, el método, la ruta y el código de respuesta, sin el cuerpo: nunca registra contraseñas. Los errores inesperados quedan en el registro, y el cliente solo recibe un mensaje genérico.

- **Transporte:** la conexión con la base va cifrada; el cifrado HTTPS entre la app y el servidor llega con el despliegue del Hito 3.
- **Datos de prueba ficticios:** las pruebas y la demostración usan usuarios inventados con correos `@ejemplo.com`. El repositorio no contiene datos de personas reales.

9. Validación, errores y pruebas

El servidor valida todo lo que recibe en dos niveles, como define el Hito 1 (sección 5.2), y cada error llega con el código que corresponde. 49 pruebas automáticas (Tabla 9) y 32 peticiones de Postman lo comprueban.

9.1 Validación en dos niveles

1. **Formato, en presentación.** Cada cuerpo y parámetro se valida contra su `schema` de Pydantic: campos obligatorios, tipos, rangos y campos no permitidos. Si algo falla, `manejadores_error.py` responde **400** con un detalle por campo, en el idioma pedido. FastAPI responde 422 por defecto en este caso; Rutia lo cambia a 400 y quita ese 422 de la documentación de `/docs`, para que el 422 signifique solo una regla de negocio.
2. **Reglas, en negocio.** Los servicios lanzan errores del dominio definidos en `core/errores.py`: `NoEncontrado`, `Conflicto`, `DatosInvalidos`, `NoAutorizado` y `DemasiadosIntentos`. La capa de negocio no conoce HTTP: es `manejadores_error.py` quien traduce cada error a 404, 409, 422, 401 o 429, siempre con el mismo formato.

Un error inesperado responde 500 con un mensaje genérico; el detalle queda solo en el registro del servidor. La app también valida antes de enviar, pero esa revisión solo ayuda al usuario: el servidor nunca confía en lo que recibe.

9.2 Pruebas automáticas con pytest

Tabla 9. *Pruebas automáticas del backend.*

Archivo	Pruebas	Qué cubre	Base de datos
`tests/test_generador.py`	10	Cantidad sugerida, tope de lugares, caso Daniel, tiempo y presupuesto, traslados y Haversine	No la necesita
`tests/test_api.py`	25	Lugares, rutas, recalcular, visitas, idiomas, el tope de 8 horas y los códigos 400, 404, 409 y 422	Real, separada
`tests/test_api_cuenta.py`	14	Registro, login, bloqueo con 429 y los derechos de la Ley 21.719 con contraseña	Real, separada

Las pruebas de la API corren contra una base PostgreSQL aparte, indicada en `TEST_DATABASE_URL`, que se recrea desde cero en cada ejecución con el mismo script de la

sección 6.1. Se ejecutan con `cd backend && pytest`; el 29 de octubre de 2026 pasaron las 49 (Anexo A).

9.3 Colección de Postman

La colección `postman/Rutia.postman_collection.json` y su entorno `Rutia.postman_environment.json` traen 32 peticiones en 6 carpetas. Cada petición comprueba su código de estado y guarda los ids que usan las siguientes.

- **Demo (5):** el orden de la presentación. Crear un lugar (201), leerlo (200), actualizarlo (200), borrarlo (204) y comprobar que ya no existe (404).
- **Rutas y usuarios (10):** registro, login, generar el caso Daniel, generar con 2 lugares, recalcular, y el CRUD de una ruta guardada.
- **Visitas (3):** marcar, listar y borrar una visita.
- **Cuenta, Ley 21.719 (5):** corregir los datos con la contraseña correcta y con una incorrecta (401), descargarlos y eliminar la cuenta, también primero con una contraseña incorrecta.
- **Errores (5):** 400, 422, 404 en inglés, 409 y 401.
- **Consultas (4):** salud del servicio, categorías y lugares en español e inglés.

10. Cambios respecto del Hito 1

Mantenemos la arquitectura y las tecnologías que firmamos en el Hito 1: monolito en tres capas, FastAPI, PostgreSQL y, para la app, React Native con Expo. El caso de Daniel (Hito 1, sección 5.3) sigue siendo el caso central y hoy es una prueba automática. En la Tabla 10 declaramos lo que cambiamos o precisamos, por qué y con qué impacto.

Tabla 10. *Cambios respecto del Hito 1.*

Cambio	Hito 1	Hito 2	Por qué	Impacto
Equipo	3 integrantes	2 integrantes: Carrera y Toro	Francisco Carvajal dejó la asignatura	Priorizamos el backend; la app móvil se completa en el Hito 3
Dónde vive la base	PostgreSQL instalado localmente (6.3)	PostgreSQL en Supabase, plan gratuito; local para las pruebas y como plan B	Una base que compartimos los dos, accesible para la demostración desde cualquier computador, sin costo	Límite de 500 MB y pausa tras una semana sin uso; conexión por el *Session pooler*

Cambio	Hito 1	Hito 2	Por qué	Impacto
Sesión y autorización	Presentación comprueba la sesión (5.2); tokens en SecureStore (6.1)	El login verifica la contraseña pero no entrega token; el cliente envía `usuario_id`; corregir, descargar y eliminar los datos piden la contraseña	El hito prioriza API, capas y persistencia; el token se integra con la app en el Hito 3	Quien conozca un id puede ver el perfil, las rutas y las visitas de ese usuario: limitación declarada (8.3 y 11)
HTTPS	Comunicación cifrada con HTTPS (5.3)	El servidor corre en local por HTTP; la conexión con la base sí va cifrada	HTTPS depende del despliegue, que llega en el Hito 3	En la demostración, el tráfico de la red local no va cifrado
Idiomas	Usuario objetivo con inglés como idioma nativo (3.1)	Toda respuesta visible en español o inglés según `Accept-Language`	El usuario objetivo es un turista que habla inglés	Textos `_es` y `_en` en lugares y categorías; errores traducidos
Categorías	Prototipo con Museos, Gastronomía y Parques (7.2)	Museos, parques, patrimonio y restaurantes	Cubrir los intereses del usuario objetivo (3.1); "Gastronomía" pasa a "Restaurantes"	5 lugares cambiaron de categoría y recreamos la base
Catálogo	9 lugares precargados (7.1), con información "esencial y verificada" (7.6)	Los mismos 9, con dirección, horario, costo, fuente y fecha de verificación; La Moneda pasa a ser visita exterior y el MAC es gratis	Verificar el catálogo era un pendiente del Hito 1 (8) y dos datos no calzaban con sus fuentes	La ruta de Daniel cuesta $4.590; las coordenadas siguen aproximadas (11)
Cantidad de lugares	"Determina cuántos lugares incluir" (5.3), sin regla	De 2 a 5, elegida por el turista o sugerida según su tiempo (5.3 de este documento)	Concretar la promesa de "pocos lugares"	Campo opcional `cantidad`; la respuesta avisa si caben menos
Orden y traslados	"Los ordena por cercanía" y estima traslados (5.2), sin método	Primero el más barato, después el más cercano que cabe; línea recta × 1,3 a 4 km/h	Un método simple, explicable y sin servicios de mapas de pago	Es una estimación, no una ruta real por calles (11)
Reordenar antes de guardar	La ruta sugerida permite reordenar o quitar paradas (7.4)	`POST /rutas/recalcular` recalcula traslados y totales en el servidor	El cliente nunca calcula totales (5.2)	Ruta nueva en la API; la app la usará en el Hito 3
Datos personales	"Editar mis datos" y política de privacidad sin funcionalidad (7.7)	Acceso, rectificación, supresión y portabilidad; las tres últimas piden la contraseña	La Ley 21.719 rige desde el 1 de diciembre de 2026	Tres rutas nuevas y borrado en cascada (8)

Cambio	Hito 1	Hito 2	Por qué	Impacto
Límite de intentos	No se mencionaba	5 contraseñas incorrectas en 15 minutos responden 429	Proteger las cuentas contra la fuerza bruta	El límite vive en memoria y se reinicia con el servidor

10.1 Pendientes que el Hito 1 dejó para este hito

La conclusión del Hito 1 (sección 8) dejó cuatro pendientes:

- ☑ **Confirmar las tecnologías mediante pruebas:** el servicio funciona con FastAPI y PostgreSQL, y lo comprueban 49 pruebas.
- ☑ **Actualizar el prototipo según las decisiones:** el prototipo web queda como evidencia del Hito 1 y su reemplazo es el inicio de la app en Expo, ya conectada a esta API (rama `feature/app-expo`).
- ☑ **Cargar y verificar el catálogo:** los 9 lugares tienen dirección, horario, costo, la página donde se verificaron y la fecha (29 de octubre de 2026). Las coordenadas siguen aproximadas, a unos 100 metros.
- ☐ **Probar el prototipo con usuarios:** pendiente para el Hito 3. Todavía no se ha probado con turistas.

11. Limitaciones conocidas y trabajo para el Hito 3

El servicio cumple el mínimo funcional del hito, pero tiene límites que conocemos y declaramos. La Tabla 11 los resume con lo que haremos en el Hito 3.

Tabla 11. *Limitaciones del Hito 2 y plan para el Hito 3.*

Limitación	Consecuencia hoy	Plan para el Hito 3
Sin token de sesión	El cliente envía `usuario_id` y el servidor confía en él	Login con token firmado (JWT) guardado en SecureStore, como decía el Hito 1
Sin despliegue ni HTTPS	El servicio corre en el computador de quien lo levanta	Publicarlo con HTTPS para que la app funcione fuera de la red local
Límite de intentos en memoria	Se reinicia junto con el servidor	Guardarlo en la base si se usa más de un servidor
Traslado estimado	Línea recta × 1,3 a 4 km/h, no una ruta real por calles	Mantenerlo y, si el tiempo alcanza, comparar con un servicio de rutas
Horarios como texto	El generador no revisa si un lugar estará abierto a la hora de la visita	Evaluar horarios estructurados por día
Catálogo de 9 lugares	Pocas opciones y coordenadas aproximadas, a unos 100 metros	Ampliar el catálogo y afinar las coordenadas con OpenStreetMap
Sin migraciones	Un cambio de esquema obliga a recrear la base	Evaluar migraciones antes de tener datos reales

Limitación	Consecuencia hoy	Plan para el Hito 3
Plan gratuito de Supabase	El proyecto se pausa tras una semana sin uso	Usarlo cada semana o tener listo el PostgreSQL local

Además, el costo de los restaurantes es un gasto estimado por persona, no un precio fijo, y los costos y horarios verificados a mano pueden cambiar. Por eso cada lugar guarda la fuente y la fecha de su verificación.

El Hito 3 completa la app móvil sobre esta API: las pantallas de la Ley 21.719, reordenar antes de guardar con `/rutas/recalcular` y elegir cuántos lugares visitar.

12. Conclusión

En esta segunda etapa pasamos de la propuesta y el prototipo del Hito 1 a un backend que funciona. Con 4 horas, $15.000 y gusto por museos y restaurantes, el servidor arma la ruta de Daniel: 4 lugares, 229 minutos y $4.590, calculados siempre con los datos de la base. La API tiene 23 rutas, separa sus tres capas de verdad, guarda los datos en PostgreSQL con un script reproducible, mantiene las credenciales fuera del repositorio y permite ejercer los derechos de la Ley 21.719. 49 pruebas automáticas lo respaldan.

Mantener un monolito en tres capas respondió al alcance actual del proyecto, a nuestro tamaño y al tiempo disponible: nos dio una estructura comprensible y fácil de probar, sin la complejidad operativa de varios servicios. No es una decisión definitiva; la revisaremos si la cantidad de usuarios, la carga o la necesidad de desplegar módulos por separado lo justifican (sección 3).

Quedan pendientes la sesión con token, el despliegue con HTTPS, la ampliación del catálogo y la prueba con usuarios. También tenemos que probar el servicio con más concurrencia, llevar el límite de intentos a un almacenamiento compartido si usamos varias instancias y completar la app móvil, para comprobar de principio a fin que las operaciones funcionan, guardan los datos y responden con errores claros. Ese es el trabajo del Hito 3.

Lo principal que aprendimos es que implementar un servicio no consiste solo en lograr que sus endpoints respondan: también hay que definir contratos claros, validar los datos, separar bien las responsabilidades y tomar decisiones de arquitectura justificadas por las necesidades reales del proyecto. Con esa base, el desafío del Hito 3 será integrar el backend con la interfaz de Rutia y validar que el usuario pueda convertir su tiempo, presupuesto e intereses en una ruta que cumpla la promesa inicial del producto.

13. Bibliografía

Equipo Rutia. (2026). *Hito 1: documento de propuesta* [Documento no publicado]. Universidad Mayor.

FastAPI. (s. f.-a). *Features*. Recuperado el 29 de octubre de 2026, de https://fastapi.tiangolo.com/features/

FastAPI. (s. f.-b). *Deployments concepts*. Recuperado el 1 de noviembre de 2026, de https://fastapi.tiangolo.com/deployment/concepts/

Ley N.° 21.719. Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. (2024, 13 de diciembre). *Diario Oficial de la República de Chile*. https://www.bcn.cl/leychile/navegar?idLey=21719

Microsoft. (s. f.). *Microservices architecture style*. Azure Architecture Center. Recuperado el 1 de noviembre de 2026, de https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices

Mozilla Developer Network. (s. f.). *422 Unprocessable Content*. MDN Web Docs. Recuperado el 29 de octubre de 2026, de https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/422

OWASP Foundation. (s. f.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. Recuperado el 29 de octubre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PostgreSQL Global Development Group. (s. f.). *5.5. Constraints*. PostgreSQL Documentation. Recuperado el 29 de octubre de 2026, de https://www.postgresql.org/docs/current/ddl-constraints.html

Psycopg. (s. f.). *Psycopg 3 documentation*. Recuperado el 29 de octubre de 2026, de https://www.psycopg.org/psycopg3/docs/

Python Software Foundation. (s. f.). *hashlib: Secure hashes and message digests*. Python Documentation. Recuperado el 29 de octubre de 2026, de https://docs.python.org/3/library/hashlib.html

SQLAlchemy. (s. f.). *ORM quick start*. SQLAlchemy 2.0 Documentation. Recuperado el 29 de octubre de 2026, de https://docs.sqlalchemy.org/en/20/orm/quickstart.html

Supabase. (s. f.-a). *Connect to your database*. Supabase Docs. Recuperado el 29 de octubre de 2026, de https://supabase.com/docs/reference/postgres/connection-strings

Supabase. (s. f.-b). *Pricing*. Recuperado el 29 de octubre de 2026, de https://supabase.com/pricing

Supabase. (s. f.-c). *Project pausing*. Supabase Docs. Recuperado el 1 de noviembre de 2026, de https://supabase.com/docs/guides/platform/free-project-pausing

14. Anexos

Anexo A: evidencia de las pruebas

Resultado de `pytest` el 29 de octubre de 2026, contra una base PostgreSQL de pruebas:

```
collected 49 items

tests/test_api.py ......................                                 [ 51%]
tests/test_api_cuenta.py ..............                                  [ 79%]
tests/test_generador.py ..........                                       [100%]

============================== 49 passed in 17.82s ==============================
```

Anexo B: declaración de uso de inteligencia artificial

El curso permite usar asistentes de inteligencia artificial si se declara qué herramienta se usó, en qué parte y cómo se verificó el resultado (Anexo C del curso). La Tabla 12 detalla el uso en este hito.

Tabla 12. *Uso de inteligencia artificial en el Hito 2.*

Herramienta	Parte del trabajo	Cómo se verificó
Claude (Anthropic), en Claude Code y en la app Claude	Código del backend: capas, generador de rutas, scripts de base de datos y pruebas, a partir de nuestras decisiones	49 pruebas automáticas, 32 peticiones de Postman y revisión del código por nosotros, que debemos poder explicar cada parte
Claude	Documentación: `README.md`, `docs/hito-2/API.md`, colección de Postman, borrador de este documento y su armado en PDF	La revisamos nosotros y abrimos cada referencia en su fuente original el 29 de octubre de 2026
Claude	Inicio de la app móvil en Expo (rama `feature/app-expo`)	20 pruebas de la app y recorrido de sus pantallas contra la API
ChatGPT (OpenAI)	Redacción y revisión de la sección 3 y de la conclusión, formato APA de las referencias y estructura de las láminas de la presentación	Contrastamos cada afirmación técnica con el código del repositorio y la documentación oficial, y abrimos cada referencia para comprobar que existe y dice lo que citamos

Las decisiones de diseño las tomamos nosotros, no la herramienta: los códigos 400 y 422, las 4 categorías, la cantidad de 2 a 5 lugares, el traslado a pie, la primera parada más barata, recalcular, los derechos de la Ley 21.719 con contraseña para corregir, descargar y eliminar, el tope de 8 horas, el límite de intentos, los datos del catálogo según sus fuentes y el `.env` en la raíz.